

Training Project: Hotel Booking System

Objective

Develop an end-to-end Service Catalog item that allows employees to book hotel rooms across global offices. You will be responsible for creating the data structure, the user interface, and the logic to ensure a seamless booking experience.

Phase 1: Data Modeling (The Foundation)

Don't hardcode locations in the form. Create a table structure to manage them.

1. **Create a Custom Table:** `u_hotel_inventory`
2. **Add Columns:**
 - **Location (Choice/String):** New York, London, Tokyo, Paris, Dubai.
 - **Room Type (Choice):** Standard, Deluxe, Suite.
 - **Amenities (List Collector/String):** Wi-Fi, Breakfast, Mini-bar, Ocean View.
 - **Price per Night (Currency).**
 - **Available Rooms (Integer).**

Phase 2: Catalog Item Creation

Create a new Catalog Item named "**Global Hotel Room Booking**".

Required Variables:

- **Requester Information:** (Reference to `sys_user`, auto-populated with the logged-in user).
- **Destination Location:** (Reference to your new `u_hotel_inventory` table).
- **Check-in Date:** (Date).
- **Check-out Date:** (Date).
- **Room Type:** (Lookup Select Box referencing the inventory table, dependent on the Location).
- **Special Requirements:** (Multi-line Text).
- **Estimated Total Cost:** (Read-only Currency field).

Phase 3: Logic & Validation (The "Brain")

Apply the following rules using **Catalog Client Scripts** and **UI Policies**:

- **Date Validation:** Prevent users from selecting a Check-in date in the past or a Check-out date that is before the Check-in date.
- **Dynamic Filtering:** When a user selects "London," the "Room Type" variable should only show rooms available in London.
- **Auto-Calculation:** Create an `onChange` script that calculates:
 $\\$\\$\\text{Total Cost} = (\\text{Price Per Night} \\times \\text{Number of Nights})\\$\\$$
- **Mandatory Fields:** Make "Special Requirements" mandatory only if the user selects a "Suite."

Phase 4: Workflow & Fulfillment

Define what happens after the "Order" button is clicked:

1. **Approval:** If the total cost is > \$1,000, send an approval request to the user's Manager.
2. **Task Creation:** Once approved, generate a **Catalog Task (SCTASK)** assigned to the "Travel Desk" assignment group to finalize the booking.
3. **Notifications:** Send an email to the requester once the task is closed/complete.

Phase 5: Technical Requirements & Best Practices

- **Naming Convention:** Use proper prefixes for all elements (e.g., **u_** for custom fields).
- **User Experience:** Use **Variable Sets** for the Requester Information so it can be reused in future catalog items.
- **Cleanup:** Ensure the form looks clean using **Containers** (Start, Split, and End).

Submission Checklist for the Associate

- ☐ Can I book a room for a date in 2023? (Should be blocked).
- ☐ Does the price update automatically when I change the room type?
- ☐ Is the "Location" data coming from the table or is it hardcoded? (Must be from the table).
- ☐ Does the Manager receive an approval email for expensive bookings?

Answer:

To ensure your fresher has a clear roadmap, here is the "Technical Reference Guide." This contains the specific logic they will need to implement the requirements effectively.

1. Data Structure (Table: **u_hotel_inventory**)

Before writing code, they must populate the data.

- **Columns:** **u_location** (Choice), **u_room_type** (Choice), **u_price_per_night** (Currency), **u_amenities** (List Collector).
 - **Sample Data:**
 - London | Deluxe | £250 | Wi-Fi, Gym, Breakfast.
 - New York | Suite | \$500 | Mini-bar, City View, Wi-Fi.
-

2. Client-Side Logic (Catalog Client Scripts)

A. Date Validation (Ensure Check-out > Check-in)

Type: `onChange` | **Variable Name:** `check_out_date`

JavaScript

```
None
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue === '') {
        return;
    }

    var checkIn = g_form.getValue('check_in_date');
    var checkOut = newValue;

    // Compare dates
    if (checkIn > checkOut) {
        g_form.showFieldMsg('check_out_date', 'Check-out date cannot be before
Check-in date.', 'error');
        g_form.clearValue('check_out_date');
    }
}
```

B. Total Cost Calculation

Type: `onChange` | **Variable Name:** `room_type` (assuming this is a lookup to the inventory table)

JavaScript

```
None
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue === '') {
        return;
    }

    var checkIn = new Date(g_form.getValue('check_in_date'));
    var checkOut = new Date(g_form.getValue('check_out_date'));

    // Calculate difference in days
    var timeDiff = checkOut.getTime() - checkIn.getTime();
    var days = Math.ceil(timeDiff / (1000 * 3600 * 24));
}
```

```

if (days > 0) {
    // Use GlideAjax to fetch the price from the custom table
    var ga = new GlideAjax('HotelBookingUtils');
    ga.addParam('sysparm_name', 'getRoomPrice');
    ga.addParam('sysparm_room_id', newValue);
    ga.getXMLAnswer(function(response) {
        var price = parseFloat(response);
        var total = price * days;
        g_form.setValue('estimated_total_cost', total);
    });
}
}

```

3. Server-Side Logic (Script Include)

To make the calculation above work, they need a **Script Include** (Client Callable).

JavaScript

```

None
var HotelBookingUtils = Class.create();
HotelBookingUtils.prototype = Object.extend(Object.prototype, {

    getRoomPrice: function() {
        var roomId = this.getParameter('sysparm_room_id');
        var gr = new GlideRecord('u_hotel_inventory');
        if (gr.get(roomId)) {
            return gr.getValue('u_price_per_night');
        }
        return 0;
    },

    type: 'HotelBookingUtils'
});

```

4. UI Policy (Dynamic Visibility)

Instead of coding, have them use a **UI Policy** for the "Special Requirements" field.

- **Conditions:** Room Type is Suite.
 - **UI Policy Actions:**
 - Special Requirements -> Mandatory: **True**
 - Special Requirements -> Visible: **True**
-

5. Workflow/Flow Designer Logic

If they are using **Flow Designer** (the modern standard), the logic should look like this:

1. **Trigger:** Service Catalog (When the item is submitted).
 2. **Action:** Get Catalog Variables (Select your Hotel variables).
 3. **If Condition:** Estimated Total Cost > 1000.
 - **Action:** Ask for Approval (Approver: Manager).
 4. **Wait for Approval:**
 - **If Approved:** * Create Catalog Task (Assignment Group: Travel Desk).
 - **If Rejected:**
 - Update RITM State to Closed Incomplete.
 - Send Email to Requester.
-

Tips for the Fresher

- **Variable Sets:** Use them for "Requester Name" and "Department" so you don't recreate them every time.
- **Reference Qualifiers:** For the Room Type variable, use an **Advanced Reference Qualifier** so it only shows rooms where u_location matches the selected location on the form.
 - *Qualifier:* javascript:'u_location=' + current.variables.destination_location;